



SyncHR

Documentation, Test Cases & Deployment Guide

Submitted to: **Professor Letterio Galletta**

A Distributed Web-Based Human Resource Management System

Built with Go (backend) and React (frontend)

Final Project — Distributed Programming for Web, IoT, and Mobile Systems

University of Florence

Syed Mohammad Askari Abidi | 7178458

Syed.abidi@edu.unifi.it

TABLE OF CONTENTS

Project Overview.....	3
Core Features	3
System Architecture.....	4
High-Level Architecture	4
Backend Architecture (Go).....	4
The WebSocket Hub (Real-Time Layer)	4
Frontend Architecture (React)	5
Database Schema	5
Technology Stack.....	6
Backend.....	6
Frontend	6
Infrastructure	6
Feature Walkthrough.....	7
Employee Dashboard	7
Apply Leave.....	8
My Leave Requests	9
View Payslip	9
HR Dashboard — Leave Requests	10
HR Dashboard — Attendance Report	11
HR Dashboard — Employee Directory	11
HR Dashboard — Send Notification.....	12
Real-Time Notification Bell	12
API Reference	12
Authentication & Users	12
Attendance	13
Leave	13
Payslip, Holidays & Notifications.....	13
Compile & Run Instructions.....	14
Option A — Docker (Recommended).....	14
Option B — Manual Setup (Native Development)	15
Step 1 — Database	15
Step 2 — Backend	15
Step 3 — Frontend	15
Creating Accounts.....	15

Deployment.....	15
Test Cases.....	16
Authentication	16
Authorization (RBAC)	17
Attendance	17
Leave Management	18
Real-Time Notifications (WebSocket)	19
Payslip	19
Known Issues Surfaced By Testing	19
Running the Automated Test Suite	20
References	20

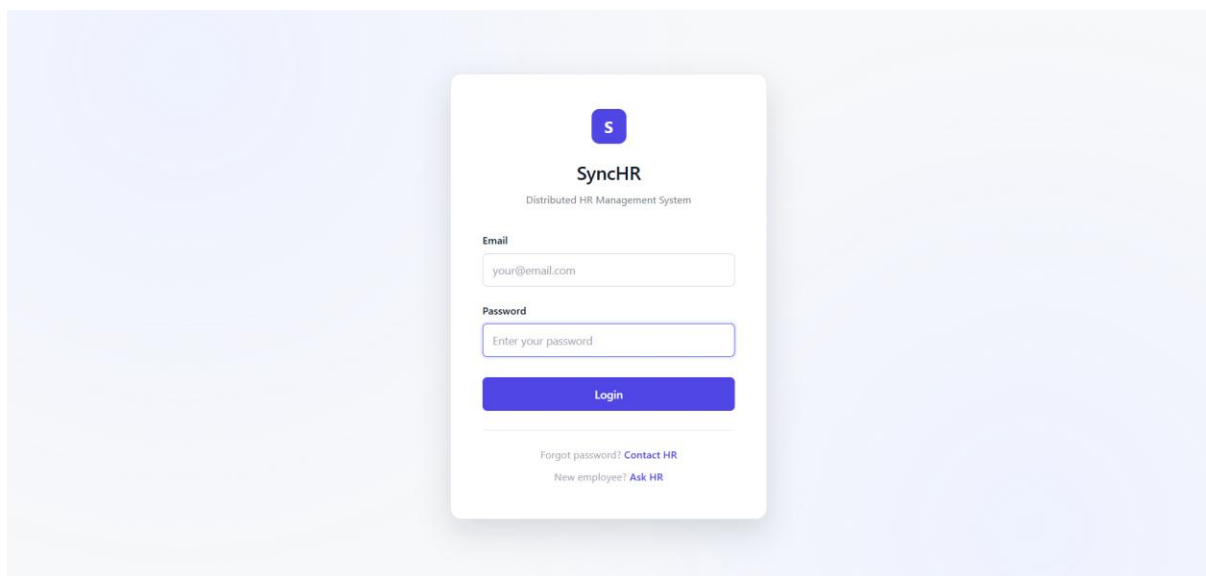
PROJECT OVERVIEW

SyncHR is a distributed, web-based Human Resource Management System that lets employees check in and out, apply for and track leave, and view payslips, while HR managers review and approve leave requests, browse attendance day-by-day, manage the employee directory, and push instant notifications to the whole workforce.

The system is built as two independently deployable services — a Go REST API backend and a React single-page application frontend — communicating over HTTP and WebSocket, backed by a PostgreSQL database. (RESTful API Design Best Practices and Guidelines, n.d.)

CORE FEATURES

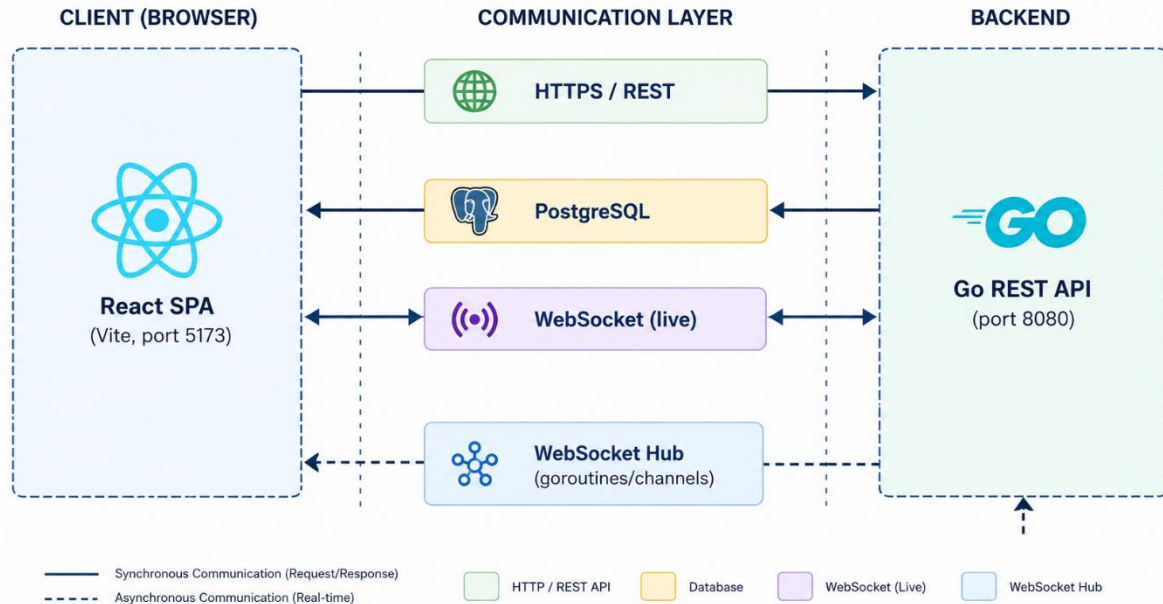
- ✓ **Authentication** — registration, login, JWT sessions, role-based routing (Employee vs HR Manager).
- ✓ **Attendance** — check in/out with a live running session timer; employees browse their own attendance by date; HR browses every employee's attendance by date with month navigation and day tabs.
- ✓ **Leave management** — employees apply for leave against a real balance and track status; HR reviews, approves, or rejects requests with notes; balances are deducted automatically on approval.
- ✓ **Payslips** — employees view monthly payslips with a full earnings/deductions' breakdown.
- ✓ **Employee directory** — HR can search and browse all employees and reset an employee's password (HR-mediated reset).
- ✓ **Real-time notifications** — leave apply/approve/reject events and HR-authored announcements persisted to the database and pushed live over a WebSocket connection to anyone online, with a notification bell showing an unread count on both dashboards.



SYSTEM ARCHITECTURE

HIGH-LEVEL ARCHITECTURE

SynchHR follows a client-server architecture with three tiers:



BACKEND ARCHITECTURE (Go)

The backend is a single Go binary built with the gorilla/mux (Gorilla Web Toolkit - Mux Package, n.d.) (Gorilla WebSocket - Real-time Communication, n.d.) router (The Go Programming Language - Official Documentation, n.d.). It is organized by domain, one handler file per feature area, all sharing a single * *sql.DB* connection pool:

- ✓ *main.go* — builds the route table (SetupRouter) and starts the HTTP server.
- ✓ *middleware.go* — JWT validation middleware, plus the WebSocket hub (the real-time notification engine).
- ✓ *auth.go* — registration, login, profile, employee directory, HR-mediated password reset.
- ✓ *attendance.go* — check-in/check-out, attendance history, HR's all-employee attendance view.
- ✓ *leave.go* — leave types, apply/approve/reject, leave balance.
- ✓ *payslip.go* — payslip listing and detail retrieval with ownership checks.
- ✓ *holiday.go* — public holiday listing.
- ✓ *notification.go* — notification persistence, retrieval, mark-as-read, HR broadcast.
- ✓ *models.go* — shared request/response/entity structs.
- ✓ *database.go* — PostgreSQL connection setup and startup schema bootstrap.

The WebSocket Hub (Real-Time Layer)

Real-time notification delivery is implemented as a classic Go "actor": a single goroutine (*Hub.run*) owns the registry of connected clients and is the only code that ever reads or mutates

it. Every other goroutine — one per connected client, plus every HTTP handler that wants to push a notification — communicates with it exclusively through channels (register, unregister, direct) rather than a shared, locked map:

```

140 }
141
142 You, 19 hours ago | 1 author (You)
143 type wsHub struct {
144     clients    map[*wsClient]bool
145     register    chan *wsClient
146     unregister  chan *wsClient
147     direct      chan wsTarget
148 }

```

Because a browser's native WebSocket API cannot set an Authorization header on the handshake, the JWT is instead passed as a `?token = query` parameter and validated with the same logic as the HTTP middleware, on a route that is deliberately public (not behind the JWT-checking subrouter). (golang-jwt - JSON Web Tokens for Go, n.d.)

FRONTEND ARCHITECTURE (REACT)

- ✓ `src/pages/` — one component per screen (Login, Employee Dashboard, HR Dashboard, Apply Leave, My Leave Requests, View Payslip).
- ✓ `src/context/` — AuthProvider and NotificationProvider (the latter owns the app-wide WebSocket connection).
- ✓ `src/hooks/` — `useAuth`, `useNotifications`, `useDateNavigator` (shared date-picker/month-nav/day-tabs logic used by both dashboards' attendance views).
- ✓ `src/components/` — ProtectedRoute (role-aware route guard) and NotificationBell.
- ✓ `src/utlis/` — shared date/time formatting helpers.
- ✓ `src/services/api.js` — the single Axios client and every REST call.

Auth and notification state are each split into a context-value module, a Provider component, and a custom hook, so Provider files export only a component (better React Fast Refresh behavior) while still exposing simple `useAuth()` / `useNotifications()` hooks to every page. (React - Official Documentation and Guide, n.d.) (Axios - Promise Based HTTP Client, n.d.)

DATABASE SCHEMA

PostgreSQL, 9 tables:

- ✓ Users
- ✓ Attendance
- ✓ break_time
- ✓ leave_types
- ✓ leave_balance
- ✓ leave_request
- ✓ public_holidays
- ✓ payslip
- ✓ notifications

Foreign keys cascade on delete (deleting a user cleans up their attendance, leave, payslip, and notification rows automatically). Full definitions are in `db/schema.sql`.

Table	Purpose
users	Employee and HR manager accounts, bcrypt-hashed passwords, role
attendance	Daily check-in/check-out records, one row per user per day
leave_types	The 5 seeded leave categories (Sick, Casual, Earned, Maternity, Paternity)
leave_balance	Remaining days per user, per leave type, per year
leave_request	Leave applications with an approval workflow (pending/approved/rejected)
payslip	Monthly salary breakdown per employee
public_holidays	Company/public holiday calendar
notifications	Persisted real-time notifications, with an is_read flag

TECHNOLOGY STACK

BACKEND

- ✓ *Go* — REST API and WebSocket server.
- ✓ *gorilla/mux* — HTTP routing.
- ✓ *gorilla/websocket* — WebSocket upgrade and framing.
- ✓ *golang - jwt/jwt (v5)* — JWT signing and verification.
- ✓ *golang.org/x/crypto/bcrypt* — password hashing.
- ✓ *lib/pq* — PostgreSQL driver.
- ✓ *joho/godotenv* — .env loading for local development.

FRONTEND

- ✓ React 19 + React Router 7
- ✓ Vite — dev server and production bundler
- ✓ Axios — REST client
- ✓ Native browser WebSocket API — real-time updates (no client library needed)
- ✓ Plain CSS with a shared design-token system (CSS custom properties), no UI framework dependency

INFRASTRUCTURE

- ✓ PostgreSQL 16
- ✓ Docker + Docker Compose — containerized deployment of all three services.
- ✓ nginx — serves the built frontend and handles client-side routing fallback.

FEATURE WALKTHROUGH

EMPLOYEE DASHBOARD

Shows today's attendance status with a live *HH:MM:SS* session timer while checked in, quick-action buttons (Check In, Check Out with a confirmation dialog, Apply Leave, My Requests, View Payslip), a day-by-day "My Attendance" browser, current leave balances across all 5 leave types, and profile information.

Welcome, John Doe
Employee Dashboard



Logout

Today's Attendance

COMPLETED

Check-in: 03:00 · Check-out: 05:47
Hours worked: 2 HR 48 MINS

Check In

Check Out

Apply Leave

My Requests

View Payslip

My Attendance

13-Jul-2026

< July 2026 >

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

Showing attendance for **Monday, 13th July 2026** (Today)

CHECK IN
03:00

CHECK OUT
05:47

DURATION
2 HR 48 MINS

STATUS
COMPLETED

Leave Balance

Sick Leave
9
days remaining

Casual Leave
10
days remaining

Earned Leave
20
days remaining

Profile Information

EMAIL
employee@example.com

DEPARTMENT
Engineering

PHONE
+39123456789

ROLE
Employee

APPLY LEAVE

A form where the employee picks a leave type (fetched live from the backend, not hardcoded), a start/end date, sees the number of days auto-calculated, and submits a reason.

Apply for Leave

Submit your leave request for approval

[Back to Dashboard](#)

Leave Type *

Select leave type

Start Date *

dd- - - - - yyyy

End Date *

dd- - - - - yyyy

Number of Days

0

Automatically calculated

Reason for Leave

Enter reason for your leave request (optional)

Submit Leave Request

Important Information

- Your HR manager will review and approve/reject your request
- You will receive a notification once your request is processed
- Check your leave balance before applying

My Leave Requests

A filterable table (All/Pending/Approved/Rejected) of the employee’s own leave history, with HR’s approval/rejection notes visible once processed.

My Leave Requests

Track all your leave applications and their status

Back to Dashboard

All Requests

Pending

Approved

Rejected

Leave Type	Start Date	End Date	Days	Status	Reason	HR Comments	Submitted
Sick Leave	2026-08-01T00:00:00Z	2026-08-01T00:00:00Z	1	APPROVED	E2E test leave	approved via e2e test	7/13/2026
Casual Leave	2026-07-31T00:00:00Z	2026-08-08T00:00:00Z	9	REJECTED	gg	no reason	7/12/2026
Casual Leave	2026-07-24T00:00:00Z	2026-07-25T00:00:00Z	2	REJECTED	no request	no	7/12/2026
Casual Leave	2026-07-14T00:00:00Z	2026-07-15T00:00:00Z	2	REJECTED	no reason	no reason	7/12/2026
Casual Leave	2026-07-13T00:00:00Z	2026-07-14T00:00:00Z	2	APPROVED	personal work	ok	7/12/2026
Casual Leave	2026-07-13T00:00:00Z	2026-07-14T00:00:00Z	2	REJECTED	personal work	no reason	7/12/2026
Casual Leave	2026-07-13T00:00:00Z	2026-07-14T00:00:00Z	2	REJECTED	personal work	no reason	7/12/2026
Casual Leave	2026-07-13T00:00:00Z	2026-07-14T00:00:00Z	2	REJECTED	personal work	no reason	7/12/2026

0
PENDING

2
APPROVED

6
REJECTED

8
TOTAL

View Payslip

A grid of monthly payslip cards with net salary at a glance, expandable into a full document-style modal with earnings, deductions, attendance summary, and net salary.

Payslips

View your monthly salary information

Back to Dashboard

July 2026

NET SALARY

₹48,000

Basic:	₹50,000
Allowances:	+₹5,000
Bonus:	+₹0
Tax:	-₹5,000
Deductions:	-₹2,000

View Full Details

June 2026

NET SALARY

₹48,000

Basic:	₹50,000
Allowances:	+₹5,000
Bonus:	+₹1,000
Tax:	-₹5,000
Deductions:	-₹2,000

View Full Details

May 2026

NET SALARY

₹48,000

Basic:	₹50,000
Allowances:	+₹5,000
Bonus:	+₹0
Tax:	-₹5,000
Deductions:	-₹2,000

View Full Details

PAYSLIP

June 2026

EMPLOYEE INFORMATION

NAME:	EMAIL:
John Doe	employee@example.com
DEPARTMENT:	MONTH/YEAR:
Engineering	June 2026

EARNINGS

Basic Salary	₹50,000
Allowances	₹5,000
Bonus	₹1,000
Total Earnings	₹56,000

DEDUCTIONS

Tax	₹5,000
Other Deductions	₹2,000
Total Deductions	₹7,000

ATTENDANCE

WORKING DAYS	LEAVE TAKEN
20	2

NET SALARY

₹48,000

This is an electronically generated payslip and requires no signature.

Close

HR DASHBOARD — LEAVE REQUESTS

HR sees every employee's leave requests as cards, filterable by status, with the leave type, dates, reason, and an Approve/Reject modal that requires notes.

0
PENDING

2
APPROVED

7
REJECTED

9
TOTAL

Pending (0)
Approved (2)
Rejected (7)
All (9)

John Doe APPROVED

ID: 3 - Engineering - Sick Leave

START DATE	END DATE	DAYS
Saturday, 1st August 2026	Saturday, 1st August 2026	1

REASON: E2E test leave

HR NOTES: approved via e2e test

Bob Wilson REJECTED

ID: 6 - Marketing - Sick Leave

START DATE	END DATE	DAYS
Wednesday, 15th July 2026	Friday, 17th July 2026	3

REASON: I am sick

HR NOTES: no

John Doe REJECTED

ID: 3 - Engineering - Casual Leave

START DATE	END DATE	DAYS
Friday, 31st July 2026	Saturday, 8th August 2026	9

REASON: gg

HR NOTES: no reason

John Doe REJECTED

ID: 3 - Engineering - Casual Leave

START DATE	END DATE	DAYS
Friday, 24th July 2026	Saturday, 25th July 2026	2

REASON:

John Doe REJECTED

ID: 3 - Engineering - Casual Leave

START DATE	END DATE	DAYS
Tuesday, 14th July 2026	Wednesday, 15th July 2026	2

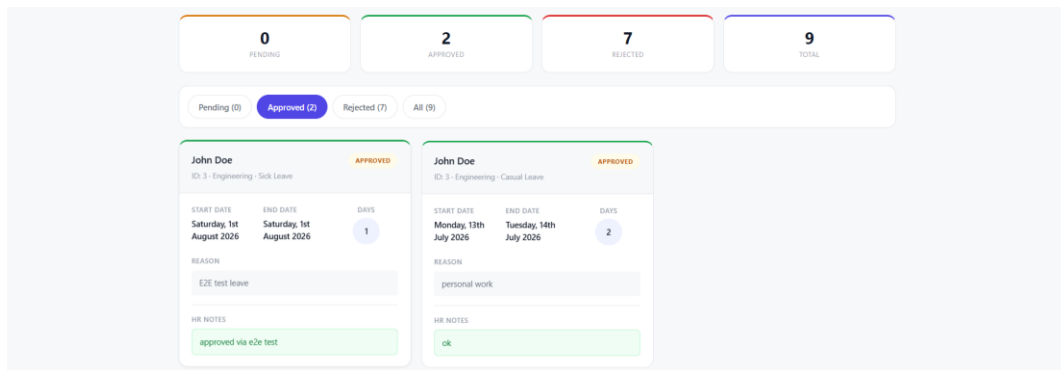
REASON:

John Doe APPROVED

ID: 3 - Engineering - Casual Leave

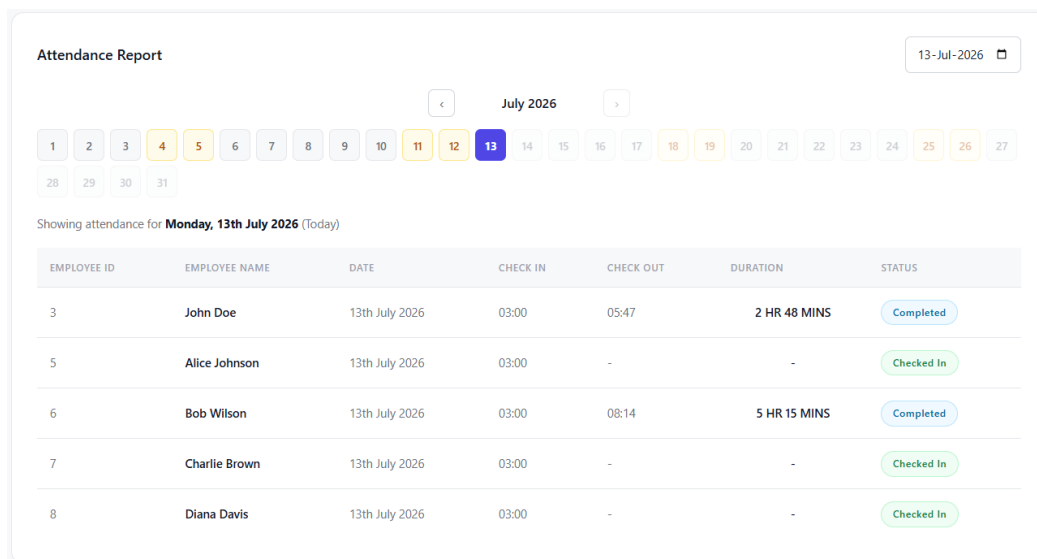
START DATE	END DATE	DAYS
Monday, 13th July 2026	Tuesday, 14th July 2026	2

REASON:



HR DASHBOARD — ATTENDANCE REPORT

A day-by-day attendance browser across all employees, with a date picker, month navigation, and day tabs (weekends highlighted, today outlined, future days disabled).



HR DASHBOARD — EMPLOYEE DIRECTORY

A searchable table of every employee, with a "Reset Password" action per row that generates a new temporary password for HR to relay to the employee.

All Employees

Search by name, email, or department...

ID	NAME	EMAIL	ROLE	DEPARTMENT	PHONE	ACTIONS
5	Alice Johnson	alice@example.com	Employee	Sales	+3911111111	Reset Password
6	Bob Wilson	bob@example.com	Employee	Marketing	+3922222222	Reset Password
7	Charlie Brown	charlie@example.com	Employee	Operations	+3933333333	Reset Password
8	Diana Davis	diana@example.com	Employee	Finance	+3944444444	Reset Password
4	Jane Smith	hr@example.com	HR Manager	HR	+39987654321	Reset Password
3	John Doe	employee@example.com	Employee	Engineering	+39123456789	Reset Password

Reset Password

Reset the password for **Diana Davis** (diana@example.com)?

Their current password will stop working immediately. You'll need to relay the new temporary password to them yourself.

Cancel
Reset Password

HR DASHBOARD — SEND NOTIFICATION

A compose form HR uses to broadcast an announcement to every employee instantly.

Send Notification to Employees

Send

REAL-TIME NOTIFICATION BELL

Present on both dashboards shows an unread-count badge and a dropdown of recent notifications, updated live over the WebSocket connection with no page refresh.

HR Manager Dashboard

Manage employee leave requests

Logout

Welcome, John Doe

Employee Dashboard

Logout

API REFERENCE

All routes are prefixed with `/api`. Protected routes require an `Authorization: Bearer <token>` header. The WebSocket route is the one exception — it is public and authenticates via `a?token=query` parameter instead.

AUTHENTICATION & USERS

Method	Path	Access	Description
POST	/auth/login	Public	Log in, returns a JWT
POST	/auth/register	Public	Create an account
GET	/users/profile	Any	Get your own profile

Method	Path	Access	Description
PUT	/users/profile	Any	Update your own profile
GET	/users/employees	HR	List every employee
PUT	/users/{id}/reset-password	HR	Generate a new temporary password for an employee

ATTENDANCE

Method	Path	Access	Description
POST	/attendance/checkin	Any	Check in for today
POST	/attendance/checkout	Any	Check out for today
GET	/attendance/history	Any	Your own full attendance history
GET	/attendance/records?date=	HR	All employees' attendance for one day (defaults to today)

LEAVE

Method	Path	Access	Description
GET	/leave/types	Any	List all leave types
POST	/leave/apply	Any	Apply for leave
GET	/leave/balance	Any	Your own leave balance
GET	/leave/requests	Any	Your own requests (employee) or everyone's (HR)
PUT	/leave/approve/{id}	HR	Approve a pending request
PUT	/leave/reject/{id}	HR	Reject a pending request

PAYSLIP, HOLIDAYS & NOTIFICATIONS

Method	Path	Access	Description
GET	/payslip/list	Any	Your own payslips (or all, for HR)

Method	Path	Access	Description
GET	/payslip/{id}	Owner or HR	One payslip's full detail
GET	/holidays	Any	Public holiday calendar
GET	/notifications	Any	Your own notifications
PUT	/notifications/{id}/read	Any	Mark one notification as read
PUT	/notifications/read-all	Any	Mark all as read
POST	/notifications/broadcast	HR	Send a notification to every employee
GET	/ws/notifications?token=	Any (query-auth)	WebSocket upgrade for live push

COMPILE & RUN INSTRUCTIONS

Two supported ways to run **SyncHR**: one command with Docker (recommended, no local Go/Node/Postgres setup needed) or running each piece natively for development. (PostgreSQL Official Documentation, n.d.)

OPTION A — DOCKER (RECOMMENDED)

Prerequisite: Docker and Docker Compose only.

docker compose up --build

This single command:

- ✓ Builds the Go backend into a small static binary in an Alpine image.
- ✓ Builds the React frontend and serves it via nginx, with the single-page-app fallback route configured so client-side routing works on refresh.
- ✓ Starts a PostgreSQL container and automatically applies *db/schema.sql* on first boot (mounted as a Postgres *init* script — no manual *psql* step).
- ✓ Wires the three services together on a private Docker network, waiting for Postgres to report healthy before starting the backend.

Once it is up: Frontend on *http://localhost:5173*, Backend API on *http://localhost:8080/api*, Postgres on *localhost:5432*. Credentials/secrets can be overridden by copying *.env.example* to *.env* at the repository root.

This flow was verified end-to-end during development: a completely fresh `docker compose up --build` (empty Postgres volume) correctly auto-applied the schema, and registration and login worked immediately through the containerized stack.

OPTION B — MANUAL SETUP (NATIVE DEVELOPMENT)

Prerequisites:

- ✓ Go (see backend/go.mod for the toolchain version)
- ✓ Node.js 18+
- ✓ Reachable PostgreSQL instance.

Step 1 — Database

```
psql -U <your_user> -d <your_db> -f db/schema.sql
```

Step 2 — Backend

```
cd backend
```

```
cp .env.example .env # fill in real DB credentials and a JWT secret
```

```
go run .
```

The API starts on *http://localhost:8080* (override with PORT in *.env*).

Step 3 — Frontend

```
cd frontend
```

```
npm install
```

```
npm run dev
```

The app starts on *http://localhost:5173*.

CREATING ACCOUNTS

db/schema.sql defines the tables only — it does not seed any user accounts. Register an account of each role via the API:

```
curl -X POST http://localhost:8080/api/auth/register \
```

```
-H "Content-Type: application/json" \
```

```
-d
```

```
{ "email": "employee@example.com", "password": "password123", "first_name": "John", "last_name": "Doe", "role": "employee", "department": "Engineering" }
```

```
curl -X POST http://localhost:8080/api/auth/register \
```

```
-H "Content-Type: application/json" \
```

```
-d
```

```
{ "email": "hr@example.com", "password": "password123", "first_name": "Jane", "last_name": "Smith", "role": "hr_manager", "department": "HR" }
```

role must be either *employee* or *hr_manager*.

DEPLOYMENT

The project ships with everything needed to run it as three containers behind Docker Compose:

- ✓ *backend/Dockerfile* (multi-stage Go build into a minimal Alpine image)
- ✓ *frontend/Dockerfile* (Vite build served by nginx, with an *nginx.conf* that falls back to index.html for client-side routes)

- ✓ Root *docker* – *compose.yml* orchestrating Postgres, the backend, and the frontend on a shared network.

For a real deployment beyond a single machine, the same three images can be pushed to any container registry and run on separate hosts, as long as:

- ✓ The frontend is built with *VITE_API_URL* pointing at the backend’s real public URL (it is a build-time argument, not a runtime environment variable, since it gets baked into the static JS bundle).
- ✓ The backend’s *DB_HOST/DB_PORT/DB_USER/DB_PASSWORD/DB_NAME* and *JWT_SECRET* are set to real values, not the Docker Compose defaults.
- ✓ CORS and the WebSocket origin check (currently permissive, for development) are tightened for production use.

Note: The WebSocket hub’s connected-client registry lives in a single process’s memory. Running more than one backend instance behind a load balancer would require a shared pub/sub layer (e.g., Redis) to fan out notifications across instances — this is a known limitation of the current single-instance design, discussed further in the accompanying report. (MDN Web Docs - Cross-Origin Resource Sharing (CORS), n.d.)

TEST CASES

27 automated Go tests cover authentication, RBAC, attendance, leave management, and the WebSocket notification hub, all passing, including under Go’s -race detector, which specifically exercises the hub’s concurrent goroutine/channel design under contention. Cases without an automated test are manual/UI checks, marked accordingly.

AUTHENTICATION

ID	Scenario	Expected Result	Automated
AUTH-01/02	Register then log in with the right credentials	200 OK, valid JWT with the correct role	Yes
AUTH-03	Log in with the wrong password	401 Unauthorized, no token issued	Yes
AUTH-04	Register with an email already in use	Registration rejected	Yes
AUTH-05	Access a protected route with no token	401 Unauthorized	Yes
AUTH-06	Access with an expired token	401 Unauthorized	Yes
AUTH-07	Access with a wrong-secret token	401 Unauthorized	Yes

ID	Scenario	Expected Result	Automated
AUTH-08	Access with an "alg: none" token	401 Unauthorized (signing method itself is validated)	Yes
AUTH-09	HR resets an employee's password	Old password stops working, new one logs in	Yes
AUTH-10	Wrong credentials on the login form	Inline error shown, no page reload	Manual
AUTH-11	Forgot password / new employee links	Opens a pre-filled mailto: to HR	Manual

AUTHORIZATION (RBAC)

ID	Scenario	Expected Result	Automated
RBAC-01	Employee calls an HR-only attendance endpoint	403 Forbidden	Yes
RBAC-02	Employee tries to approve leave	403 Forbidden	Yes
RBAC-03	Employee tries to broadcast a notification	403 Forbidden	Yes
RBAC-04	HR views the employee directory	200 OK, full list	Manual
RBAC-05	Employee navigates to the HR Dashboard URL	Redirected to their own dashboard	Manual
RBAC-06	Employee tries to reset a password	403 Forbidden	Yes
RBAC-07	Two tabs, different accounts logged in	Stale tab syncs to the newest login	Manual

ATTENDANCE

ID	Scenario	Expected Result	Automated
ATT-01/03	Full check-in → check-out cycle	200 OK both times, status becomes checked_out	Yes

ID	Scenario	Expected Result	Automated
ATT-02	Duplicate check-in same day	400 Bad Request	Yes
ATT-04	Check out without checking in	400 Bad Request	Yes
ATT-05	View own attendance history	200 OK, only own records	Yes
ATT-06	HR views a specific day, all employees	200 OK, filtered to that date	Manual
ATT-07	Live session timer while checked in	HH:MM:SS increments every second	Manual
ATT-08	Check-out confirmation dialog	Dialog appears before submitting	Manual

LEAVE MANAGEMENT

ID	Scenario	Expected Result	Automated
LEAVE-00	Fetch all leave types	All 5 seeded types returned	Yes
LEAVE-01	Apply for leave within balance	201 Created, status pending	Yes
LEAVE-02	Apply for more days than balance allows	400 Bad Request	Yes
LEAVE-03/04	HR approves a request	200 OK, balance decremented, employee notified	Yes
LEAVE-05/06	HR rejects a request	200 OK, status rejected, employee notified	Yes
LEAVE-07	HR sees a new request live	Live WebSocket push, no refresh needed	Manual (verified via E2E script)
LEAVE-08	HR filters by status	Only matching requests shown	Manual
LEAVE-09	Leave type shown by name	Real name from a DB join, not a guessed ID	Yes

REAL-TIME NOTIFICATIONS (WEBSOCKET)

ID	Scenario	Expected Result	Automated
NOTIF-01/02	Connect with no/invalid token	401, handshake refused	Yes
NOTIF-03	Notification targeted at one user	Only that user's socket receives it	Yes
NOTIF-04	Notification targeted at a role	Only sockets with that role receive it	Yes
NOTIF-05	One client disconnects	Other clients unaffected, hub keeps running	Yes
NOTIF-06	20 clients messaged concurrently	All 20 receive their message, no data race	Yes
NOTIF-07	HR broadcasts an announcement	Persisted for every employee + live push	Manual (verified via E2E script)
NOTIF-08	Reading a notification	is_read flips true, unread count drops	Manual + server logic covered
NOTIF-09	Dropped connection reconnects	Automatic retry within a few seconds	Manual

PAYSLIP

ID	Scenario	Expected Result	Automated
PAY-01/02	View own payslip list and detail	200 OK, only own data returned	Manual
PAY-03	Try to view another employee's payslip by ID	Access denied	Manual

KNOWN ISSUES SURFACED BY TESTING

Writing this suite surfaced real, previously-undiscovered bugs, all now fixed and regression-covered:

- ✓ Login accepted no real passwords for API-registered accounts (a plaintext comparison instead of bcrypt.CompareHashAndPassword). Every account created through the register endpoint was permanently unable to log in.
- ✓ A timezone mismatch between Go's local time and Postgres's timezone-less TIMESTAMP column produced negative attendance durations (e.g. "-2.00 hours" for a near-instant check-in/check-out).

- ✓ The frontend hardcoded leave type names to IDs 1-3, silently making 2 of the 5 seeded leave types (Maternity, Paternity) impossible to select and mis-rendering their names anywhere they appeared.

RUNNING THE AUTOMATED TEST SUITE

Integration tests need a reachable Postgres database (configured via backend/.env). Each test registers its own throwaway account and deletes it afterward via *t.Cleanup*, so the suite is safe to run repeatedly without leaving data behind. If no database is reachable, DB-backed tests are skipped rather than failing the whole run.

```
C:\Windows\System32\cmd.exe
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 222, Email: test_employee_1783979450272917900@example.com, Role: employee
--- PASS: TestLeave_GetLeaveTypes (0.12s)
=== RUN TestLeave_RequestIncludesLeaveTypeName
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 223, Email: test_employee_1783979450397154900@example.com, Role: employee
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 223, Email: test_employee_1783979450397154900@example.com, Role: employee
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 223, Email: test_employee_1783979450397154900@example.com, Role: employee
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 223, Email: test_employee_1783979450397154900@example.com, Role: employee
--- PASS: TestLeave_RequestIncludesLeaveTypeName (0.13s)
=== RUN TestLeave_ApplyInsufficientBalance_Rejected
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 224, Email: test_employee_1783979450530623300@example.com, Role: employee
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 224, Email: test_employee_1783979450530623300@example.com, Role: employee
--- PASS: TestLeave_ApplyInsufficientBalance_Rejected (0.12s)
=== RUN TestLeave_FullApprovalFlow
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 225, Email: test_employee_1783979450655138600@example.com, Role: employee
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 225, Email: test_employee_1783979450655138600@example.com, Role: employee
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 226, Email: test_hr_manager_178397945077440400@example.com, Role: hr_manager
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 225, Email: test_employee_1783979450655138600@example.com, Role: employee
2026/07/13 23:50:50 🟡 JWT Middleware - Extracted UserID: 225, Email: test_employee_1783979450655138600@example.com, Role: employee
--- PASS: TestLeave_FullApprovalFlow (0.25s)
=== RUN TestLeave_RejectFlow
2026/07/13 23:50:51 🟡 JWT Middleware - Extracted UserID: 227, Email: test_employee_1783979450904614300@example.com, Role: employee
2026/07/13 23:50:51 🟡 JWT Middleware - Extracted UserID: 227, Email: test_employee_1783979450904614300@example.com, Role: employee
2026/07/13 23:50:51 🟡 JWT Middleware - Extracted UserID: 228, Email: test_hr_manager_1783979451025518500@example.com, Role: hr_manager
2026/07/13 23:50:51 🟡 JWT Middleware - Extracted UserID: 227, Email: test_employee_1783979450904614300@example.com, Role: employee
--- PASS: TestLeave_RejectFlow (0.25s)
=== RUN TestParseJWT_ValidToken
--- PASS: TestParseJWT_ValidToken (0.00s)
=== RUN TestParseJWT_WrongSecret
--- PASS: TestParseJWT_WrongSecret (0.00s)
=== RUN TestParseJWT_ExpiredToken
--- PASS: TestParseJWT_ExpiredToken (0.00s)
=== RUN TestParseJWT_MalformedToken
--- PASS: TestParseJWT_MalformedToken (0.00s)
=== RUN TestParseJWT_RejectsNoneAlgorithm
--- PASS: TestParseJWT_RejectsNoneAlgorithm (0.00s)
PASS
ok      github.com/askariabidi/SynchR  5.054s
D:\DP\Synchr\backend\
```

REFERENCES

Axios - Promise Based HTTP Client. (n.d.). Retrieved from <https://axios-http.com/>

golang-jwt - JSON Web Tokens for Go. (n.d.). Retrieved from <https://github.com/golang-jwt/jwt>

Gorilla Web Toolkit - Mux Package. (n.d.). Retrieved from <https://github.com/gorilla/mux>

Gorilla WebSocket - Real-time Communication. (n.d.). Retrieved from <https://github.com/gorilla/websocket>

MDN Web Docs - Cross-Origin Resource Sharing (CORS). (n.d.). Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

PostgreSQL Official Documentation. (n.d.). Retrieved from <https://www.postgresql.org/docs/>

React - Official Documentation and Guide. (n.d.). Retrieved from <https://react.dev/>

RESTful API Design Best Practices and Guidelines. (n.d.). Retrieved from <https://restfulapi.net/>

The Go Programming Language - Official Documentation. (n.d.). Retrieved from <https://go.dev/doc/>